

Daily Sprint Report

Date: 2026-03-28

Project: Legal AI Platform

Scope Covered Today

Today focused on moving the project from a backend-heavy prototype into a product-shaped platform with:

- a stabilized backend foundation
- an internal legal workspace frontend
- voice intake and transcription
- transcript-to-consultation workflow extraction
- a separate public client portal with no internal AI or agent exposure

Sprint Progress Completed Today

Sprint 1. Backend Stabilization

Completed work:

- fixed the broken dependency file
- added missing AI/runtime dependencies
- aligned Docker services with backend expectations by adding Redis
- made config safer for frontend integration and optional OpenAI use
- added '.env.example'

Main files:

- 'requirements.txt'
- 'docker-compose.yml'
- 'backend/core/config.py'
- '.env.example'

Sprint 2. Internal Case Workspace Frontend

Completed work:

- created a dedicated internal frontend app
- added login and registration
- added case list and case selection
- added client creation
- added case creation
- added document upload
- added document intelligence panel
- added evidence-aware legal copilot chat

Main files:

- 'frontend/src/App.tsx'
- 'frontend/src/lib/api.ts'
- 'frontend/src/types.ts'
- 'frontend/src/styles.css'

Backend support added:

- CORS middleware
- document list and document detail routes for frontend use

Sprint 3. Voice Intake & Interaction v1

Completed work:

- added voice recording persistence model
- added voice upload API
- added browser microphone recording support in the frontend
- added transcription service integration
- added transcript storage and transcript display in the internal workspace

Main files:

- 'backend/models/voice_recording.py'
- 'backend/api/voice.py'
- 'backend/api/voice_schema.py'
- 'backend/services/ai/transcription_service.py'

Sprint 4. Transcript Intelligence & Intake Workflow

Completed work:

- added transcript intake extraction service
- extracted booking intent, urgency, legal area, preferred schedule, issue summary, and contact details
- added consultation request persistence model
- added internal UI panel to generate consultation requests from selected transcripts
- added consultation review panel in the internal workspace

Main files:

- 'backend/models/consultation_request.py'
- 'backend/api/consultations.py'
- 'backend/api/consultation_schema.py'
- 'backend/services/ai/transcript_intake_service.py'

Separate Client Portal

Completed work:

- created a separate 'client-portal' frontend app
- ensured clients do not access internal agents, evidence panels, or internal legal workspace tools
- added safe public intake submission endpoint
- added public status lookup by reference
- enabled public form submission with:
 - client details
 - issue summary
 - preferred schedule
 - voice note
 - supporting document

Main files:

- 'client-portal/src/App.tsx'
- 'client-portal/src/lib/api.ts'
- 'client-portal/src/styles.css'
- 'backend/api/public.py'
- 'backend/api/public_schema.py'

Architectural Outcome

The project now has two clearly separated surfaces:

1. Internal Legal Workspace

For:

- lawyers
- admins
- assistants

Capabilities:

- case workspace
- document intelligence
- evidence-aware copilot
- voice intake review
- transcript-to-intake conversion

2. Client Intake Portal

For:

- external clients only

Capabilities:

- submit consultation request
- upload voice note

- upload supporting file
- provide scheduling preference
- check request status using a public reference

Restricted from:

- internal models
- agent orchestration controls
- evidence panels
- internal case operations

Verification Completed

Verified today:

- backend Python compilation succeeded after changes
- frontend source files were checked for text encoding issues
- client portal and internal workspace source trees were created successfully

Not verified in this environment:

- frontend runtime build, because Node.js is not installed in the current environment
- live OpenAI transcription calls, because they depend on local runtime/API availability

Recommended Next Step

The next major sprint should be:

Multi-Agent Legal AI

Recommended first agent set:

- Intake Agent
- Retrieval Agent
- Case Reasoning Agent
- Drafting Agent
- Verifier Agent

Why now:

- the business workflows now exist
- the client and internal surfaces are separated
- the agents can orchestrate real value instead of wrapping incomplete logic

Deliverables Produced Today

- backend stabilization improvements
- internal legal workspace frontend
- voice intake and transcript handling
- transcript-to-consultation workflow extraction

- separate client portal
- architecture and sprint documentation

End-of-Day Status

The project has moved from a backend-first prototype toward a platform with:

- real internal product UI
- real client-facing UI
- real voice intake flow
- real consultation extraction workflow
- a cleaner separation between operational users and external clients